

DOSSIER DE RENDU — PROJET FINAL

Projet : Lency — SaaS de gestion communautaire

Auteur : Timothée Van Den Bosch

Date : 14 avril 2026

Domaine	URL
Production	https://lency.net
Staging	https://staging.lency.net

1. Application déployée et fonctionnelle (/8 pts)

Accessibilité

L'application est accessible en production sur **<https://lency.net>** et en pré-production sur **<https://staging.lency.net>**.

Le déploiement est effectué via **Vercel** (PaaS), connecté au dépôt GitHub [tim-vdb/Lency](#) :

- Branche **main** → déployée automatiquement sur [lency.net](#) (production)
- Branche **staging** → déployée automatiquement sur [staging.lency.net](#) (staging)

Dernier déploiement staging vérifié : commit [790b386](#), état **READY**, région **iad1** (Washington DC, East).

Base de données

La base de données est un **PostgreSQL serverless** hébergé sur **Neon.com**, connectée via **Prisma ORM**. Elle est opérationnelle sur les deux environnements avec des instances séparées (production / staging).

Projet récupéré depuis GitHub

Le projet est lié au dépôt privé [github.com/tim-vdb/Lency](#). Chaque push sur une branche surveillée déclenche automatiquement un nouveau build et déploiement sur Vercel.

Front correctement servi

Le frontend est une application **Next.js 15 (App Router)** compilée via **Turbopack**. Vercel sert les assets statiques depuis son CDN et les 33 routes dynamiques sont exposées via des **fonctions serverless Node.js 24.x** :

Domaine fonctionnel	Routes serverless
Auth	/api/auth/[...all]
Articles	/api/articles , /api/articles/[articleId]

Domaine fonctionnel	Routes serverless
Badges	<code>/api/badges, /api/badges/[badgeId]</code>
Categories	<code>/api/categories, /api/categories/[categoryId]</code>
Emails	<code>/api/emails/welcome</code>
Events	<code>/api/events, /api/events/[eventId]</code>
MapLocations	<code>/api/mapLocations, /api/mapLocations/[mapLocationId]</code>
Newsletter	<code>/api/newsletterSubscribers, /api/newsletterSubscribers/[newsletterSubscriberId]</code>
Posts & Comments	<code>/api/posts, /api/posts/[postId], /api/posts/[postId]/comments, /api/posts/[postId]/comments/[commentId]</code>
Projects	<code>/api/projects, /api/projects/[projectId]</code>
Resources	<code>/api/resources, /api/resources/[resourceId]</code>
Spots	<code>/api/spots, /api/spots/[spotId]</code>
Subscriptions	<code>/api/subscriptions, /api/subscriptions/[subscriptionId]</code>
Upload	<code>/api/uploadthing</code>
UserConfig	<code>/api/userConfig, /api/userConfig/[userConfigId]</code>
Users	<code>/api/users, /api/users/[userId], /api/users/change-password, /api/users/confirm-email-change, /api/users/verify-email-change</code>

2. Qualité de l'hébergement / architecture (/4 pts)

Choix d'hébergement : PaaS — Vercel

Vercel a été retenu comme hébergeur **PaaS** pour sa compatibilité native avec Next.js, sa gestion automatique du scaling, du CDN et du SSL, et sa simplicité d'intégration CI/CD avec GitHub.

Architecture multi-environnements

```

GitHub (tim-vdb/Lency)
├─ branch main      → Vercel → lency.net          (Production)
└─ branch staging   → Vercel → staging.lency.net    (Staging)

```

Chaque environnement dispose de :

- Son propre projet Vercel (**lency** / **lency-staging**)
- Sa propre base de données Neon.com
- Ses propres variables d'environnement injectées via **Doppler**

Architecture claire

- **Frontend** : Next.js 15, App Router, Turbopack
 - **Backend** : API Routes Next.js → Fonctions serverless Node.js 24.x (33 routes)
 - **Base de données** : Neon.com (PostgreSQL serverless) + Prisma ORM
 - **Gestion des secrets** : Doppler (projets isolés prod / staging)
 - **Emails** : Resend (transactionnel via support@mail.lency.net) + Proton Mail SMTP (social@lency.net)
 - **Fichiers** : UploadThing (</api/uploadthing>)
 - **Sous-domaines** :
 - lency.net — production
 - staging.lency.net — pré-production
 - mail.lency.net — envoi d'emails (SPF / DKIM / DMARC configurés)
-

3. Sécurité et bonnes pratiques (/4 pts)

HTTPS

HTTPS activé automatiquement sur tous les domaines via les certificats SSL gérés par Vercel (Let's Encrypt). Aucune connexion HTTP non chiffrée n'est possible.

Secrets non exposés

Les variables d'environnement (clés API, chaînes de connexion DB, secrets d'auth) sont **exclusivement gérées via Doppler**. Elles ne sont jamais committées dans le dépôt GitHub ([.env.local](#) présent dans [.gitignore](#)). Doppler injecte les secrets au moment du build Vercel selon l'environnement (production ou staging).

Debug désactivé en production

Le mode debug Next.js est désactivé en production. Les logs d'erreur sont disponibles uniquement via le dashboard Vercel (runtime logs) et non exposés publiquement.

Accès sécurisé

L'authentification est gérée via **Better Auth** ([/api/auth/\[...all\]](/api/auth/[...all])), avec sessions sécurisées. L'accès au dashboard Vercel est protégé par compte Vercel (2FA disponible). Aucun accès SSH nécessaire (architecture PaaS — pas de serveur à gérer).

4. Exploitabilité / maintenance (/2 pts)

Mise à jour depuis GitHub

Tout déploiement est déclenché automatiquement par un push GitHub sur la branche correspondante. Aucune intervention manuelle requise pour déployer.

Logs et supervision

- **Build logs** : disponibles dans le dashboard Vercel pour chaque déploiement
- **Runtime logs** : fonctions serverless loguées en temps réel sur Vercel (filtrage par niveau, source, status code)
- **Base de données** : supervision via le dashboard Neon.com (connexions, queries, métriques)

Procédure d'accès serveur

Architecture PaaS — pas de serveur à administrer directement. L'accès se fait via :

1. Dashboard Vercel : <https://vercel.com/dashboard>
2. Dashboard Neon.com pour la base de données
3. Dashboard Doppler pour les secrets
4. CLI Vercel (**vercel**) pour les opérations avancées

5. Dossier de rendu (/2 pts)

Solution retenue

(à compléter)

Étapes de déploiement

(à compléter)

Difficultés rencontrées

(à compléter)

Justification des choix

(à compléter)